

IFGOIANO – CAMPUS URUTAI SI		 INSTITUTO FEDERAL GOIANO Campus Urutai
ED I	3º Matutino	Júnio César de Lima
Lista de Exercícios 3 Listas ligadas (encadeadas)		

Exercício 0: Análise de Eficiência de Busca

Qual o número médio de nós acessados ao procurar um elemento que **pertence** à lista desordenada? E no caso de uma lista ordenada, qual a vantagem na média de acessos quando o elemento não está na lista?

Exercício 1: Conversão Vetor-Lista (Importação)

Escreva um método que crie uma lista encadeada a partir de um vetor, mantendo a ordem original dos elementos.

Exercício 2: Conversão Lista-Vetor (Exportação)

Escreva um método que copie uma lista encadeada para um vetor, mantendo a ordem original dos elementos.

Exercício 3: Concatenação com Preservação (Cópia)

Faça um método para concatenar duas listas (L1 e L2) em uma nova lista (L3), que deverá ser retornada. Os elementos de L1 devem preceder os de L2. L1 e L2 devem permanecer intactas.

Exercício 4: Remoção em Lote (Validação de Prefixo)

Faça um método para remover os n primeiros elementos de uma lista. O método deve retornar true se a remoção de todos os “n” elementos for concluída com sucesso, ou false caso a lista possua menos de “n” elementos (neste caso, nenhum elemento deve ser removido).

Exercício 5: Busca de Extremos

Faça um método para buscar a posição e o valor do maior elemento da lista, o qual deve ser retornado.

Exercício 6: Permutação de Elementos

Faça um método que receba dois valores (chaves) e realize a troca de posição entre os nós que contêm esses elementos na lista. O método deve validar se ambos os elementos existem e retornar true somente se a troca for concluída com sucesso.

Exercício 7: Filtragem de Dados

Considere uma lista contendo números inteiros positivos. Implemente um método que percorra a lista e retorne a lista contendo apenas os números pares encontrados. Não usar diretamente o método remove.

Exercício 8: Cálculo Estatístico

Considere uma lista contendo números inteiros positivos. Implemente um método que calcule e retorne a média aritmética de todos os valores presentes na lista. Caso a lista esteja vazia, o método deve tratar a exceção ou retornar zero.

Exercício 9: Verificação de Ordenação

Implemente um método que verifique se os elementos da lista estão organizados de forma lógica. O método deve retornar true se a lista estiver estritamente crescente ou estritamente decrescente, e false caso os elementos estejam desordenados.

Exercício 10: Clonagem de Estrutura

Crie um método que realize a cópia fiel de uma lista L1 para uma nova lista L2. Todos os nós de L2 devem ser instâncias novas (novas alocações de memória), garantindo que alterações futuras em L2 não afetem o conteúdo de L1.

Exercício 11: Cópia com Distinção

Implemente um método que copie os elementos de uma lista L1 para uma nova lista L2, garantindo que L2 não possua elementos repetidos.

Exercício 12: Inversão por Cópia

Escreva um método que receba uma lista L1 e gere uma nova lista L2 contendo os mesmos elementos, porém em ordem inversa.

Exercício 13: Divisão de Lista (Split)

Implemente um método que receba uma lista e um valor n. O método deve localizar a primeira ocorrência do valor n e dividir a estrutura: a partir do nó imediatamente após n, todos os elementos subsequentes devem formar uma nova lista, que será retornada pelo método. Se n não for encontrado ou for o último elemento, retorne uma lista vazia.

Exercício 14: Concatenação com Reuso de Nós (Eficiência)

Escreva um método que receba duas listas, L1 e L2, e as concatene em uma única estrutura, retornando o ponteiro para a nova lista.

Exercício 15: Comparação de Estruturas

Implemente um método que verifique se duas listas são identicamente iguais. Para serem iguais, elas devem possuir o mesmo número de elementos, na mesma ordem e com o mesmo conteúdo. O método deve retornar um valor booleano.

Exercício 16: Inversão In-Place (Desafio de Ponteiros)

Escreva um método que inverta a ordem dos elementos de uma lista sem usar nenhuma lista ou estrutura auxiliar (vetores, pilhas, etc.). Você deve realizar a inversão apenas alterando o campo prox dos nós existentes.

Exercício 17: Operação de União

Dadas duas listas L1 e L2, implemente o método União, que cria uma terceira lista L3 contendo todos os elementos que aparecem em L1 ou em L2. Certifique-se de que L3 não contenha elementos duplicados, mesmo que eles apareçam em ambas as listas originais.

Exercício 18: Operação de Intersecção

Dadas duas listas L1 e L2 (sem repetidos internamente), implemente o método Intersecção, que cria uma terceira lista L3 contendo apenas os elementos que estão presentes em ambas as listas simultaneamente.

Exercício 19: Limpeza de Ocorrências

Em uma lista que permite elementos repetidos, implemente um método que receba um valor X e elimine todas as células que contenham esse valor. O método deve funcionar corretamente mesmo que o elemento X esteja no início, no meio ou no fim da lista.

Exercício 20: Medição Recursiva

Implemente uma função recursiva para calcular o número total de nós de uma lista encadeada.

Exercício 21: Igualdade Iterativa

Implemente um método não recursivo para verificar se duas listas são iguais. Elas devem ser consideradas iguais se possuírem o mesmo tamanho e os mesmos elementos na mesma ordem.

Exercício 22: Igualdade Recursiva

Implemente um método recursivo para verificar se duas listas são iguais. A função deve comparar os nós correspondentes de cada lista e realizar a chamada recursiva para o próximo par de nós.

Exercício 23: Impressão Recursiva

Faça um método recursivo para imprimir todos os elementos de uma lista encadeada na tela.

Exercício 24: Ordenação de Estrutura (Sorting)

Considerando uma lista que armazena números inteiros, escreva um método que ordene a lista em ordem crescente.

Exercício 25: Decomposição por Paridade (Split)

Dada uma lista encadeada de números inteiros, escreva um método que a transforme em duas novas listas: a primeira contendo apenas os elementos pares e a segunda apenas os ímpares.

Exercício 26: Intercalação por Posição (Merge)

Implemente um método que receba duas listas (L1 e L2) e retorne uma terceira lista L3 intercalando os elementos pela posição: 1º de L1, 1º de L2, 2º de L1, 2º de L2, e assim por diante.

Exercício 27: Intercalação Ordenada (Sorted Merge)

Implemente um método que receba duas listas já ordenadas e retorne uma nova lista contendo os elementos de ambas, mantendo a ordem crescente no resultado final.

Exercício 28: Contagem Condicional

Implemente um método que receba um valor inteiro n como parâmetro e retorne a quantidade de nós na lista que possuem um valor estritamente maior que n .

Exercício 29: Somatório de Elementos

Escreva um método que percorra toda a lista de inteiros e retorne a soma total de todos os valores contidos nos nós. Caso a lista esteja vazia, o retorno deve ser 0.

Exercício 30: Gestão de Produtos

Considere que o tipo abaixo está armazenada na sua lista. Faça um método para buscar o produto de menor preço. O método deve retornar se a operação foi possível ou não.

Produto
-codigo: int -nomeProd: String -preco: double -qtdeEstoque: int
+Produto() +Produto(codigo:int,nomeProd:String,preco:double,qtdeEstoque:int) +getters&setters()

Exercício 31: Modelagem de Polígonos (Geometria Computacional)

Um polígono pode ser representado como uma **lista encadeada**, onde cada nó armazena as coordenadas (x, y) de um vértice e aponta para o próximo. Para fechar o polígono, o último vértice deve apontar obrigatoriamente para o primeiro.

- **Tarefa:** Projete uma classe Poligono que utilize uma lista encadeada interna.
- **Operações:**
 - 1. `inserirVertice(double x, double y)`: Adiciona um novo ponto ao polígono.
 - 2. `removerAresta(int indice)`: Remove o vértice naquela posição, conectando automaticamente o anterior ao próximo.

- 3. `desenhar()`: Um método que percorra a lista imprimindo as conexões (ex: "Ligando (0,0) a (0,5)").

Exercício 32: Gerenciamento de Memória (Destruição)

Implemente uma rotina para a remoção completa de uma lista, garantindo que todos os nós sejam desconectados para facilitar a coleta de lixo (*Garbage Collection*).

- **Assinatura:** `void Remove_list(Lista L)`.

Exercício 33: Laboratório de Integração de TAD Lista

Escreva um programa principal (*Main*) que utilize o seu TAD Lista para realizar as seguintes operações de manipulação de dados:

- Instanciação: Crie quatro objetos de lista independentes: L1, L2, L3 e L4.
- Geração de Dados (L1): Insira sequencialmente na lista L1 dez números inteiros gerados aleatoriamente (no intervalo entre 0 e 99).
- Geração de Dados (L2): Repita o processo de geração randômica para a lista L2 (também com 10 elementos).
- União de Estruturas: Concatene as listas L1 e L2, armazenando o resultado na lista L3. (Escolha entre a concatenação por cópia ou por reuso, conforme os exercícios anteriores).
- Processamento Inverso: Popule a lista L4 com todos os elementos presentes em L3, porém dispostos na ordem inversa.
- Auditoria de Dados: Exiba o conteúdo formatado das quatro listas no console para conferência dos resultados.

Exercício 36: Simulação de Strings com Listas Encadeadas

Considere que uma String pode ser representada como uma lista encadeada, onde cada nó contém um único caractere (`char`). Implemente um conjunto de métodos para manipular essas estruturas.

- `strcreate(char[] str)`**: Converte um vetor de caracteres (`char[]`) em uma lista encadeada. Cada caractere do vetor deve ocupar um nó da lista. O método deve retornar o ponteiro para o nó de cabeçalho da lista criada.
- `strcnvlc(Lista lista)`**: Operação inversa: converte uma lista encadeada de caracteres de volta para um vetor de caracteres (`char[]` ou `String` do Java).
- `strvrfyl(Lista l1, Lista l2)`**: Determina a **primeira posição** (índice) na lista l1 que contém um caractere que **não existe** em nenhum nó da lista l2. Retorna o índice inteiro ou -1 caso todos os caracteres de l1 estejam presentes em l2.
- `strsstr(Lista l1, int i1, int i2)`**: Extrai uma **substring** da lista l1. A nova lista deve começar no índice i1 e ter o tamanho i2. O método retorna o cabeçalho da nova lista, mantendo a lista original l1 intacta.
- `strreplace(Lista l1, int pos, int tam, Lista l2)`**: (*Refatorado para clareza*): Substitui um trecho da lista l1 (que começa em pos e tem comprimento tam) pelo conteúdo integral da lista l2. A operação deve resultar em uma nova lista, sem alterar a l1 original.
- `strcmpl(Lista l1, Lista l2)`**: Compara lexicograficamente (ordem alfabética) duas strings representadas por listas. O retorno segue o padrão da função `strcmp` da linguagem C:
 - **-1**: Se a string l1 for menor que l2.
 - **0**: Se as strings forem identicamente iguais.
 - **1**: Se a string l1 for maior que l2.